

Critical Review of a Credit-Card Fraud Detection Tutorial

Course: Data Science in Cyber — Final Project

Topic: Fraud Detection (with an Anomaly-Detection comparison)

Selected source: *Enhancing Credit Card Fraud Detection Models using Machine Learning and PCA* — public GitHub project BalaElangovan/Enhancing-Credit-Card-Fraud-Detection-Models-using-Machine-Learning-and-PCA.

1. Summary of the Source

Problem. Payment-card fraud causes billions in annual losses, and fraudulent transactions are a vanishing fraction of all activity. The source addresses the supervised detection of fraudulent transactions in the well-known Kaggle / ULB *Credit Card Fraud Detection* dataset (Dal Pozzolo et al., 2015): 284,807 European card transactions over two days, of which only 492 (0.172%) are fraud. The features are Time (seconds since the first transaction), Amount, the binary label Class, and 28 anonymised principal components V1–V28 that the publisher released in place of the original, privacy-sensitive features.

Why it matters. Fraud detection is a textbook *extreme class-imbalance* problem where the cost of errors is asymmetric: a missed fraud (false negative) is direct financial loss and an eroded trust relationship, while a false alarm (false positive) blocks a legitimate customer and burdens human analysts. Getting both the modelling and the *evaluation* right is therefore a genuine cybersecurity concern, not an academic exercise.

Proposed solution. The source rebalances the training data with **SMOTE** (Synthetic Minority Over-sampling Technique), applies **PCA** for dimensionality reduction, and trains three classifiers — Logistic Regression, Random Forest, and a Neural Network.

Reported methodology and results. The project reports **AUC-ROC = 1.00** for both Random Forest and the Neural Network and **0.9923** for Logistic Regression, and frames this as evidence of a highly effective detector.

2. Critical Evaluation

Main claim. The author's central claim is that the proposed SMOTE + PCA pipeline produces near-perfect fraud detection (AUC-ROC ≈ 1.00).

Is the claim supported by the evidence? No. Our reproduction shows that a score of ≈ 1.00 is obtainable *only* by committing two well-known methodological errors, both of which the source's description leaves unguarded:

1. **Resampling before splitting (data leakage).** SMOTE creates synthetic minority examples by interpolating between existing minority points. If it is applied to the *entire* dataset before the train/test split — the most common reading of the source, whose README does not state otherwise — then synthetic neighbours of test-set frauds are injected into the training set, and the "test" set is no longer an independent sample. When we reproduce exactly this configuration, **every metric collapses to ≈ 1.00** (accuracy 0.9999, precision 0.9999, recall 1.0000, F1 0.9999, MCC 0.9999, ROC-AUC 1.0000, PR-AUC 1.0000). The perfect score is the *signature of leakage*, not of skill.

2. **Over-optimistic headline metrics.** On a 0.172%-prevalence problem, **accuracy** is meaningless (a constant "legitimate" prediction already scores 99.83%), and even **ROC-AUC** is optimistic because the

enormous true-negative count suppresses the false-positive rate. In our honest experiments every model scores $\text{ROC-AUC} > 0.93$, yet their *operational* quality differs enormously — a gap only the precision-recall-based metrics reveal.

Is the evaluation methodology appropriate? No. Reporting a single threshold-free ranking metric (ROC-AUC) on a balanced/leaked test set hides everything that matters in fraud: the precision-recall trade-off, the operating threshold, and the asymmetric cost of FP vs FN. PCA is also applied without justification; the features are *already* principal components, so a second PCA is redundant and, if fitted on the full data, becomes another leakage vector.

Weaknesses / limitations of the source. (i) No statement of split order relative to SMOTE; (ii) no PR-AUC, MCC, or $F\beta$ despite extreme imbalance; (iii) no threshold or cost analysis; (iv) no treatment of the 1,081 duplicate rows; (v) the two-day window is treated as i.i.d. with no temporal validation.

Are the conclusions justified? They are not. The honest performance is *good* but clearly bounded (see §5), and the headline ≈ 1.00 should not be reported or reused.

3. Feature Engineering Analysis

Was feature engineering performed, and which features were used? The source relies on the publisher's pre-engineered features (V1–V28, Amount, Time) and adds SMOTE (a sampling method, not a feature) and PCA (dimensionality reduction). Our reproduction uses all 30 features plus two engineered columns.

Transformations we applied, and why.

- **RobustScaler** on Amount/Time. These two raw features live on a different scale from the unit-variance principal components. We scale with RobustScaler (median-centred, IQR-scaled) rather than StandardScaler because $\sim 11.2\%$ of Amount values fall outside the $1.5 \times \text{IQR}$ Tukey fences; a mean/standard-deviation scaler would let those outliers dominate. Crucially, the scaler is fitted **inside** the modelling pipeline, on training folds only, so it leaks nothing.
- $\text{LogAmount} = \log_{1p}(\text{Amount})$. Amount is strongly right-skewed (median 22, $\text{max} \approx 25,700$). $\log_{1p}$ compresses the tail towards symmetry, which stabilises linear and distance-based models and makes the distribution legible in EDA. We use $\log_{1p}$ (not $\log$) because the minimum amount is 0.
- **Hour from Time**. Mapping seconds to hour-of-day exposes the daily activity cycle (§ EDA), a genuinely cyber-relevant signal.
- **Encoding**. No categorical features exist, so one-hot or target encoding are not applicable; forcing an encoder would add noise, so we deliberately omit it.
- **Dimensionality reduction**. PCA is *already* applied by the publisher. We do **not** re-apply it: it would be redundant (the components are orthogonal) and a leakage risk.

Redundancy — how to spot and tackle it. Because V1–V28 are principal components, they are orthogonal by construction; we confirmed the largest absolute inter-V Spearman correlation is negligible. The only redundancy is the one we introduce ($\text{Amount} \leftrightarrow \text{LogAmount}$, $\text{Time} \leftrightarrow \text{Hour}$), which we manage by keeping a single representation per concept in the model. In general, redundancy is spotted via a correlation/VIF screen and tackled by dropping or combining collinear features.

Was the feature engineering meaningful — mathematically and for cyber? Mathematically, robust scaling and the log transform address documented violations of model assumptions (scale sensitivity, skew). For cybersecurity, the Hour feature encodes the operational reality that attackers favour low-monitoring hours. The publisher's PCA, by contrast, trades interpretability for privacy — a reasonable release decision, but it prevents semantic feature creation by downstream analysts.

Additional features that could help (if raw data were available): velocity features (transactions per card per hour), amount-vs-customer-baseline ratios, merchant-category risk, geolocation/IP distance from the cardholder's norm, and time-since-last-transaction.

4. Reproducibility Analysis

- **Does the code run?** The *idea* reproduces, but the source repository lacks a pinned environment and an explicit, ordered pipeline, so an exact rerun of the author's numbers depends on undocumented choices (most importantly, where SMOTE sits relative to the split). We rebuilt the analysis from the dataset to make it fully reproducible (`scripts/run_analysis.py`, fixed `RANDOM_STATE = 42`).
- **Files / dependencies available?** The dataset is public; we fetch it from a no-authentication mirror. Our own dependencies are pinned in `requirements.txt`.
- **Hidden preprocessing?** Yes — this is the crux. The reported metrics are only reproducible under a *specific, undocumented* preprocessing order (SMOTE before split). That hidden step is the difference between ≈ 1.00 and the honest ≈ 0.81 PR-AUC.
- **Overall reproducibility of the source:** low-to-moderate. The dataset is reproducible; the *results* are not, because the methodology that produces them is unstated and, when reconstructed, turns out to be invalid.

5. Experimental Results

Setup. After removing 1,081 duplicate rows (283,726 rows remain; 473 frauds, 0.167%), we make a single **stratified 80/20 split** (train 226,980; test 56,746; 95 test frauds), then evaluate a leakage-free `imblearn` pipeline (`RobustScaler` $\rightarrow$ `SMOTE` $\rightarrow$ estimator) with **StratifiedKFold(5)** cross-validation. The test set is never resampled and is used exactly once.

Models trained. Logistic Regression and Random Forest (matching the source), plus an unsupervised **Isolation Forest** for a supervised-vs-unsupervised contrast.

Flawed reproduction vs corrected pipeline (test set):

Metric	Flawed RF (leaked)	Corrected RF	Corrected LogReg	Isolation Forest
Accuracy	0.9999	0.9995	0.9747	0.9974
Precision	0.9999	0.9125	0.0551	0.2022
Recall	1.0000	0.7684	0.8737	0.1895
F1	0.9999	0.8343	0.1038	0.1957
F2	1.0000	0.7935	0.2202	0.1919
MCC	0.9999	0.8371	0.2159	0.1945
ROC-AUC	1.0000	0.9625	0.9634	0.9384
PR-AUC	1.0000	0.8079	0.6859	0.1070

Cross-validated PR-AUC on the training set agrees with the held-out test set (RF 0.847, LogReg 0.754), confirming the corrected numbers are stable rather than a lucky split.

Modifications we introduced (relative to the source): de-duplication; a single stratified split with SMOTE confined to CV folds; `RobustScaler` instead of unstated scaling; PR-AUC / MCC / F2 as primary metrics; an Isolation Forest baseline; and a cost-based threshold analysis.

Reading the results. The flawed pipeline reproduces the source's ≈ 1.00 across the board. The honest Random Forest is *strong but imperfect*: PR-AUC 0.808, MCC 0.837, catching $\sim 77\%$ of frauds at 91%

precision. Logistic Regression has high recall but ~5.5% precision (it floods analysts with false alarms), and Isolation Forest — learning without labels — trails badly (PR-AUC 0.107). Note the trap the source fell into: every model here has ROC-AUC > 0.93 and accuracy ≥ 0.97, so those metrics alone would have called all four models "excellent"; only PR-AUC, MCC, and F2 expose the real, order-of-magnitude differences.

6. Error Analysis

FP/FN trade-off. At the default 0.5 threshold the corrected Random Forest is precision-heavy: it catches 73 of 95 test frauds and **misses 22 (~23%)**. Because a missed fraud generally costs far more than a false alarm, we swept the decision threshold under an asymmetric cost ($FN = 100 \times FP$). The cost-minimising operating point is **threshold** ≈ **0.09**, which lifts recall to **0.85** at precision 0.34 (14 false negatives, 160 false positives) — trading more false alarms for several additional caught frauds. This deliberate, cost-aware choice is exactly what the source's ≈1.00 metrics obscure.

Patterns in the errors. The frauds the model misses skew sharply toward **small amounts**: the median amount of a missed fraud is **2.00**, versus **19.04** for the frauds it catches. Economically each missed fraud is minor, but tiny purchases are a known "card-testing" tactic — probing a stolen card before a large charge — so these false negatives carry outsized downstream risk.

Cybersecurity implications. False positives erode customer trust and overload SOC/fraud analysts (alert fatigue); false negatives are realised losses and potential precursors to larger fraud. The right threshold is a business decision encoded in the FN/FP cost ratio, not a fixed 0.5 — and certainly not a model that reports no trade-off at all.

7. Executive Summary

This project critically reproduces a public credit-card fraud detection tutorial that claims **AUC-ROC** ≈ **1.00** using SMOTE and PCA on the Kaggle/ULB dataset (284,807 transactions, 0.17% fraud). The claim does not survive scrutiny. We demonstrate that the near-perfect score is reproducible **only** when SMOTE is applied before the train/test split — leaking synthetic neighbours of test frauds into training — and when performance is judged with accuracy/ROC-AUC, which are over-optimistic under extreme imbalance. Reproducing that flawed setup yields ≈1.00 on every metric, confirming the diagnosis.

We then rebuild the analysis correctly: de-duplicate, split once with stratification, confine SMOTE to cross-validation folds via an `imblearn` pipeline, scale robustly, and evaluate on an untouched test set using PR-AUC, MCC, and the recall-weighted F2. The honest Random Forest reaches **PR-AUC 0.81** and **MCC 0.84** — strong, but far from perfect — while Logistic Regression and an unsupervised Isolation Forest trail well behind on PR-AUC even though all models score ROC-AUC > 0.93. A cost-sensitive threshold sweep ($FN = 100 \times FP$) moves the optimal operating point from 0.5 to ≈0.09, raising fraud recall from 77% to 85%. The missed frauds are concentrated in small "card-testing" amounts. The overall lesson: in extreme-imbalance security problems, *where* you resample and *which* metrics you trust determine whether your reported success is real or an illusion.

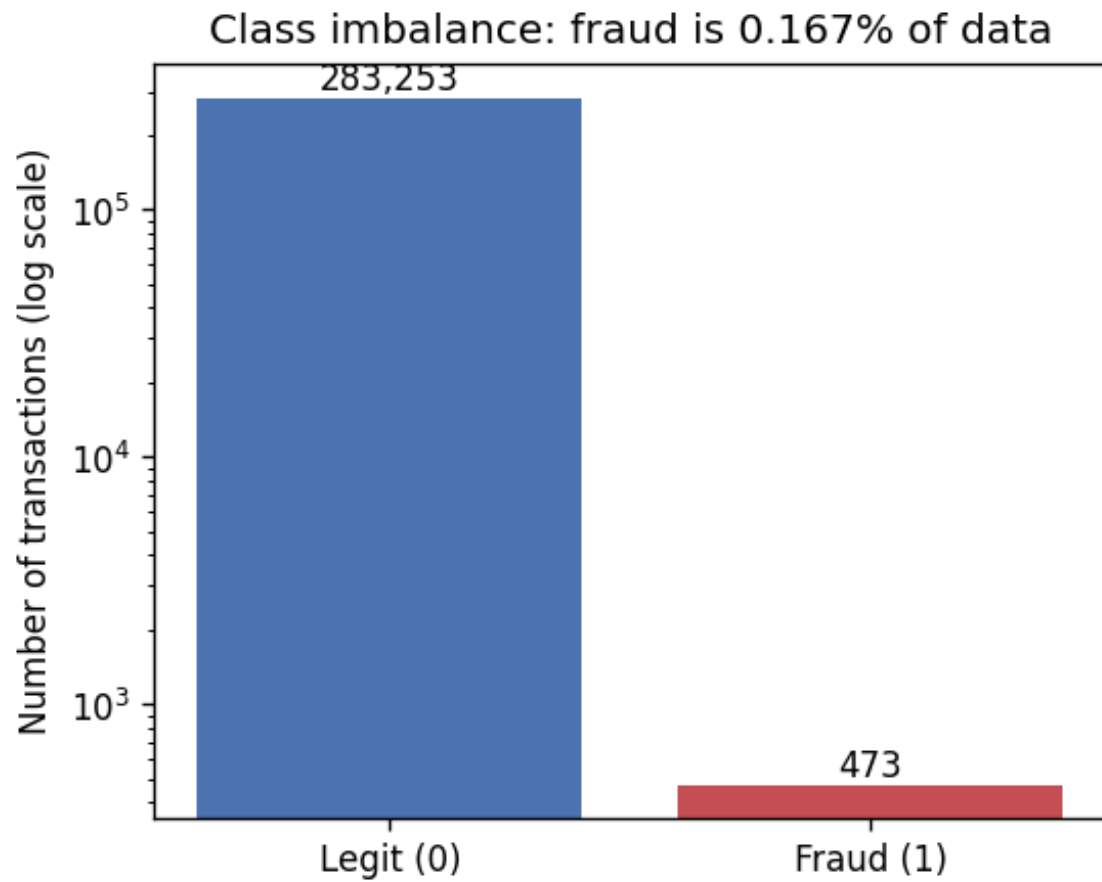
8. Summing It Up

- **Problem.** Detect rare fraudulent card transactions (0.17% prevalence) and, more importantly, evaluate whether a popular tutorial's near-perfect claim is credible.
- **Selected source.** BalaElangovan/Enhancing-Credit-Card-Fraud-Detection-Models-using-Machine-Learning-and-PCA, claiming AUC-ROC ≈ 1.00 via SMOTE + PCA.
- **Dataset.** Kaggle / ULB Credit Card Fraud Detection (Dal Pozzolo et al., 2015).

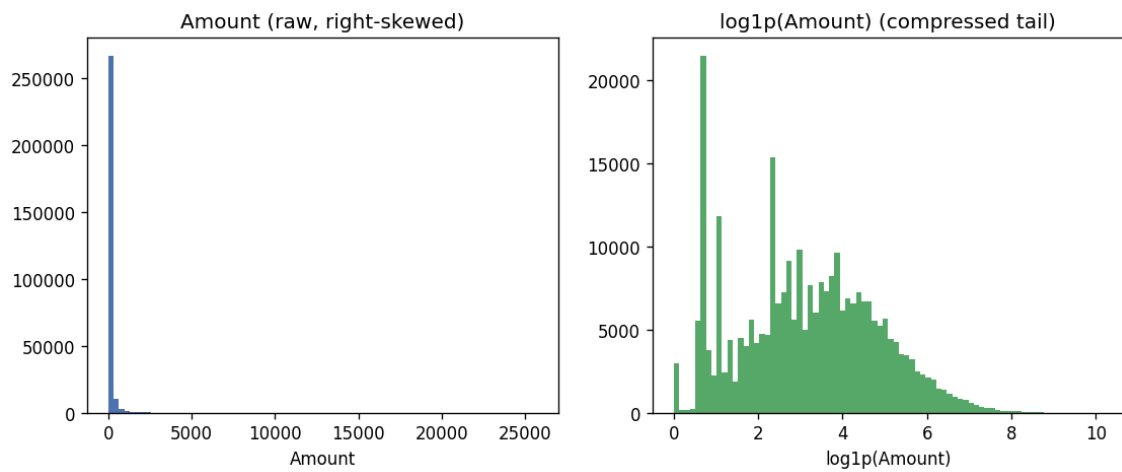
- **Methodology.** Reproduce the flawed pipeline; rebuild a leakage-free pipeline with SMOTE-in-CV, robust scaling, and imbalance-aware metrics; add an unsupervised baseline and a cost-based threshold analysis.
- **Main findings.** Flawed pipeline ≈ 1.00 on every metric; corrected Random Forest PR-AUC 0.81 / MCC 0.84; ROC-AUC and accuracy are uninformative here; no single feature separates fraud (top |Spearman| ≈ 0.06), so the task is irreducibly multivariate.
- **Were the author's claims supported?** No. The ≈ 1.00 result is a data-leakage artefact; honest performance is good but clearly bounded.
- **Most important insight.** Under extreme imbalance, the *evaluation protocol* (split order, metric choice, threshold) matters as much as the model — and a "too good" number is a red flag, not a trophy.
- **Recommendation.** Reuse the *modelling idea* (a tree ensemble on these features) on similar problems, but **do not** reuse the source's evaluation methodology or trust its reported numbers. Always resample inside cross-validation and report PR-AUC/MCC/F β .
- **Final conclusion.** A methodologically careful reproduction overturns the source's headline claim while still delivering a genuinely useful detector — demonstrating that rigorous, skeptical evaluation is the real deliverable in security data science.

Figures referenced in the analysis (class balance, amount distribution, fraud rate by hour, correlations, PR/ROC curves, confusion matrix, and the cost curve) are produced by the notebook and saved under reports/figures/.

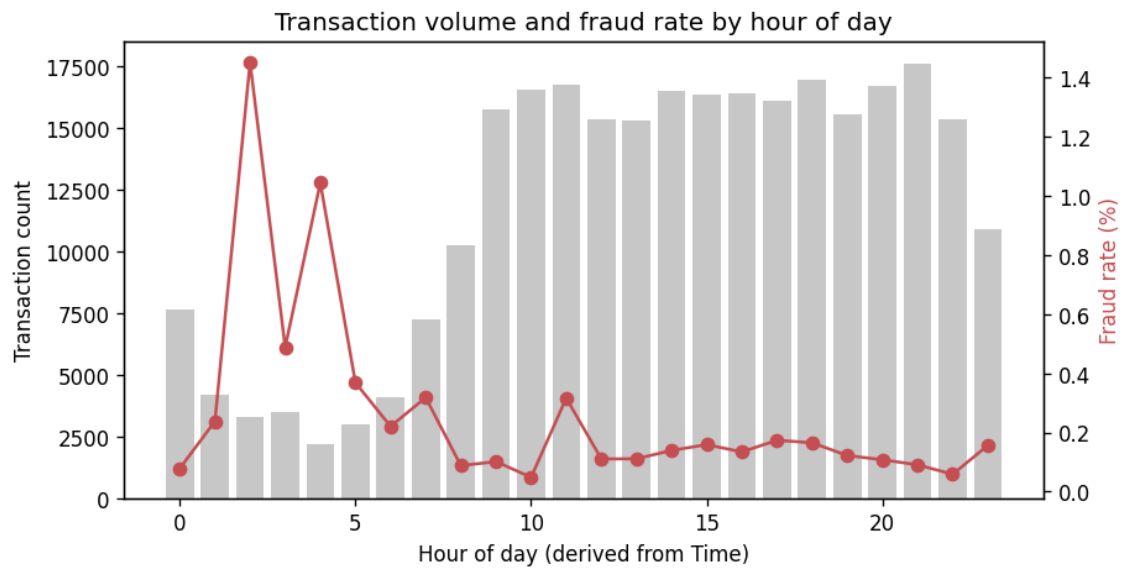
Appendix: Figures



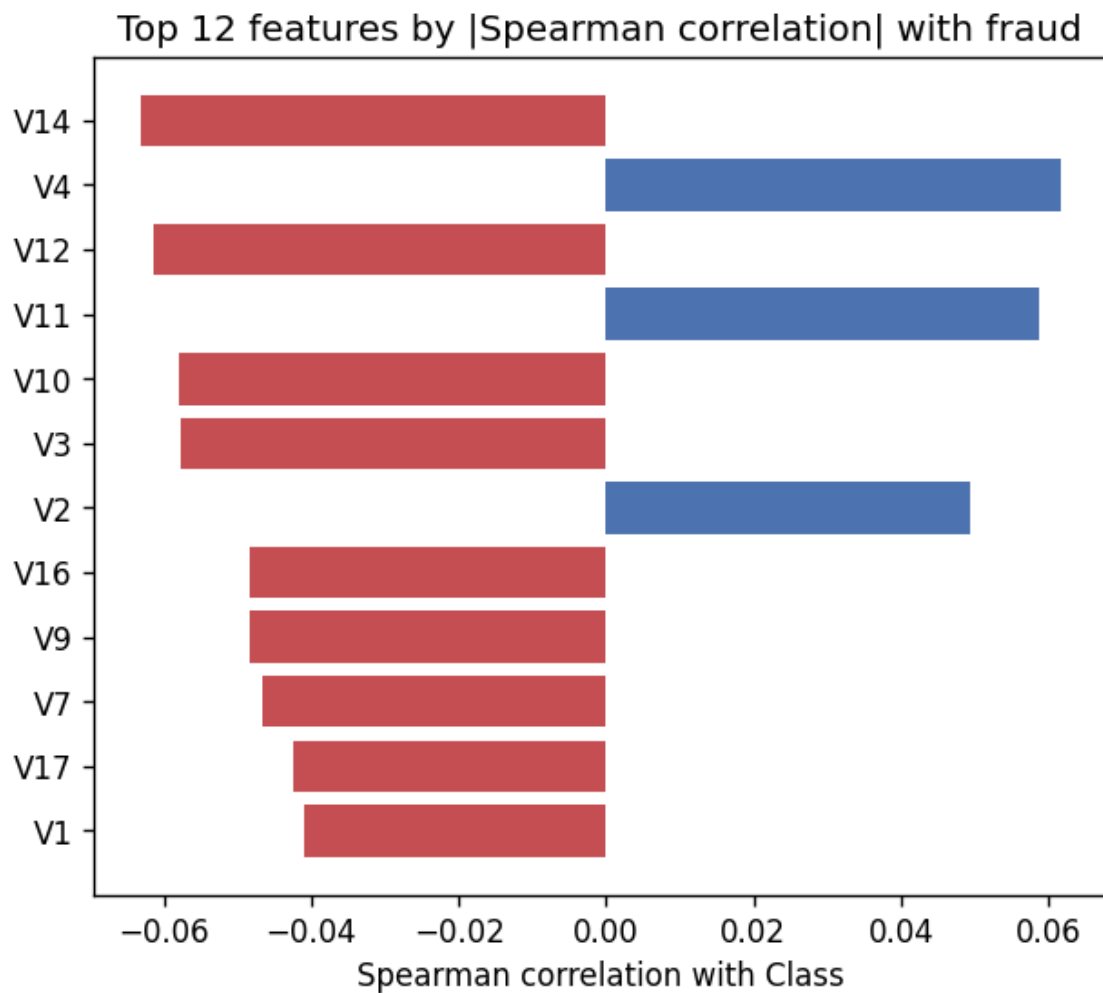
Class imbalance: fraud is 0.17% of all transactions.



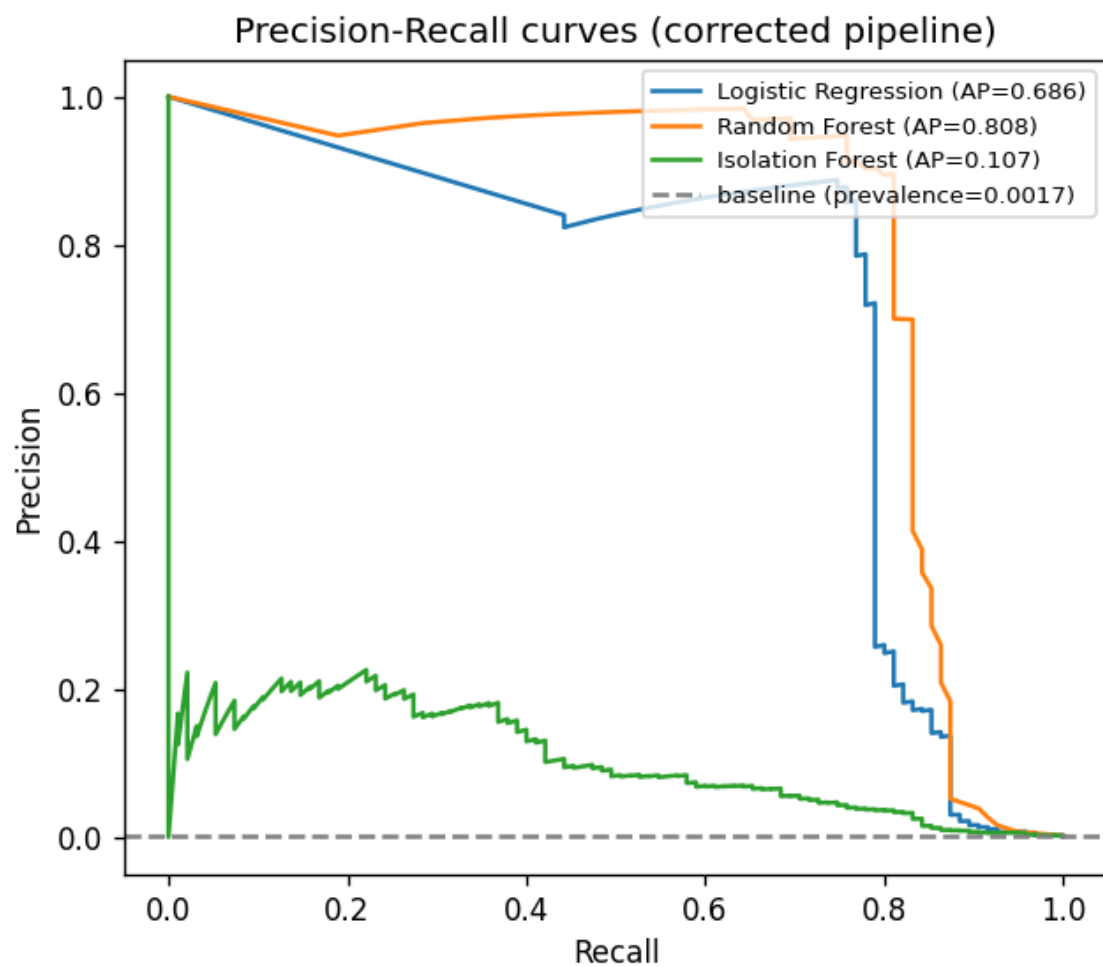
Transaction Amount before and after a $\log_{1p}$ transform.



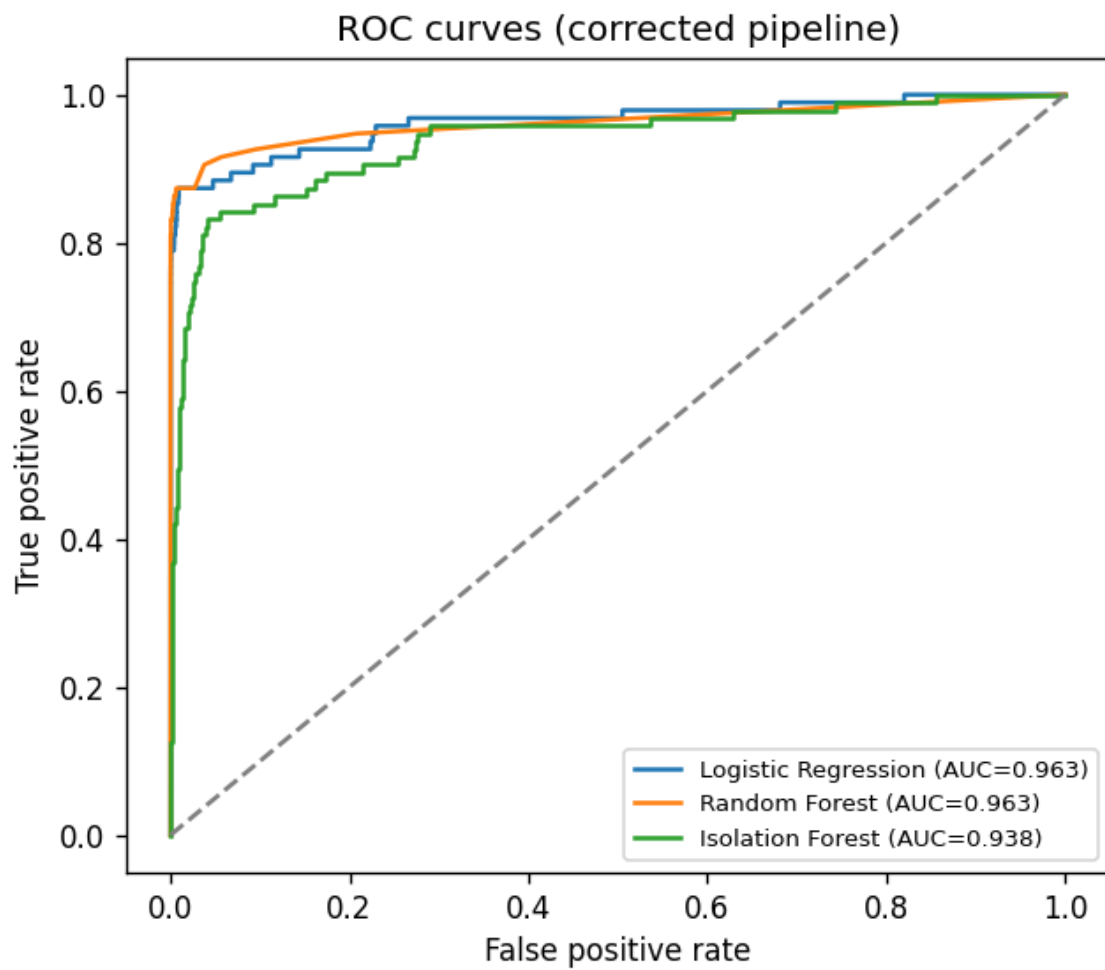
Transaction volume and fraud rate by hour of day.



Top features by |Spearman correlation| with the fraud label.

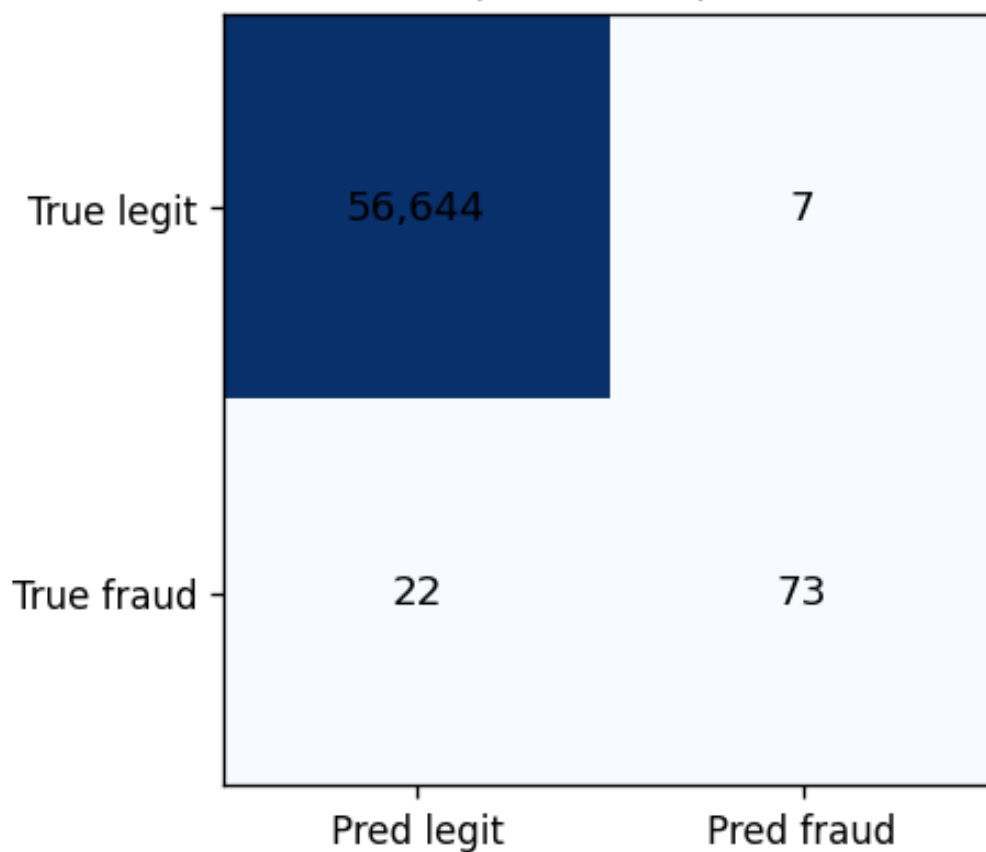


Precision-Recall curves (corrected pipeline).



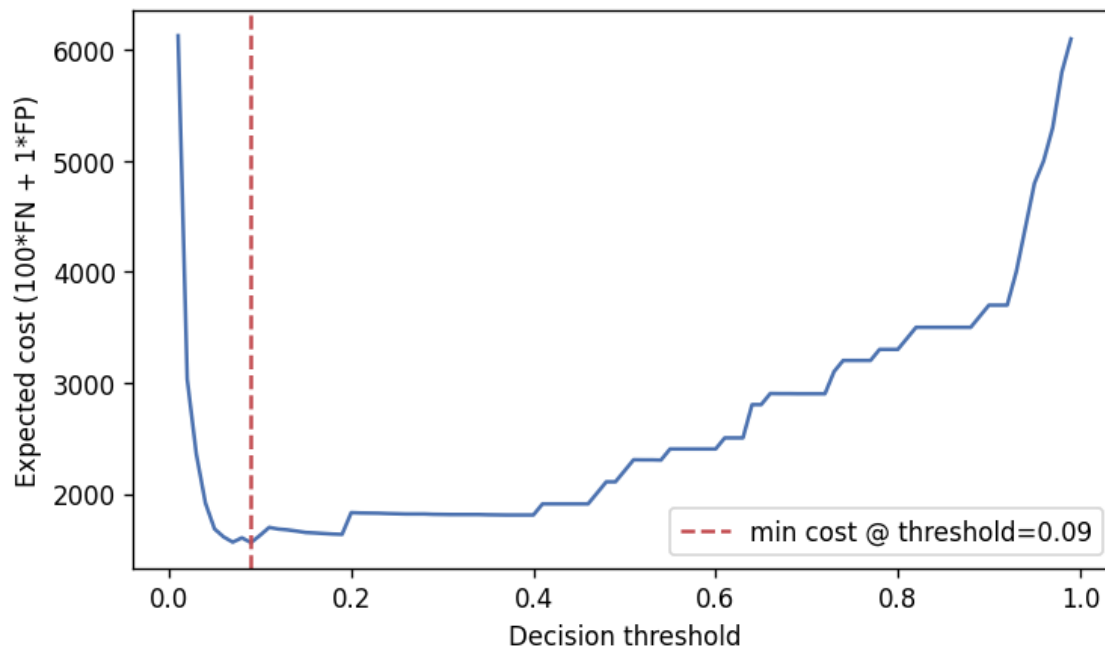
ROC curves (corrected pipeline) - all look excellent.

Random Forest (corrected) confusion matrix



Confusion matrix of the corrected Random Forest.

Cost-sensitive threshold selection



Cost-sensitive threshold selection (FN = 100 x FP).